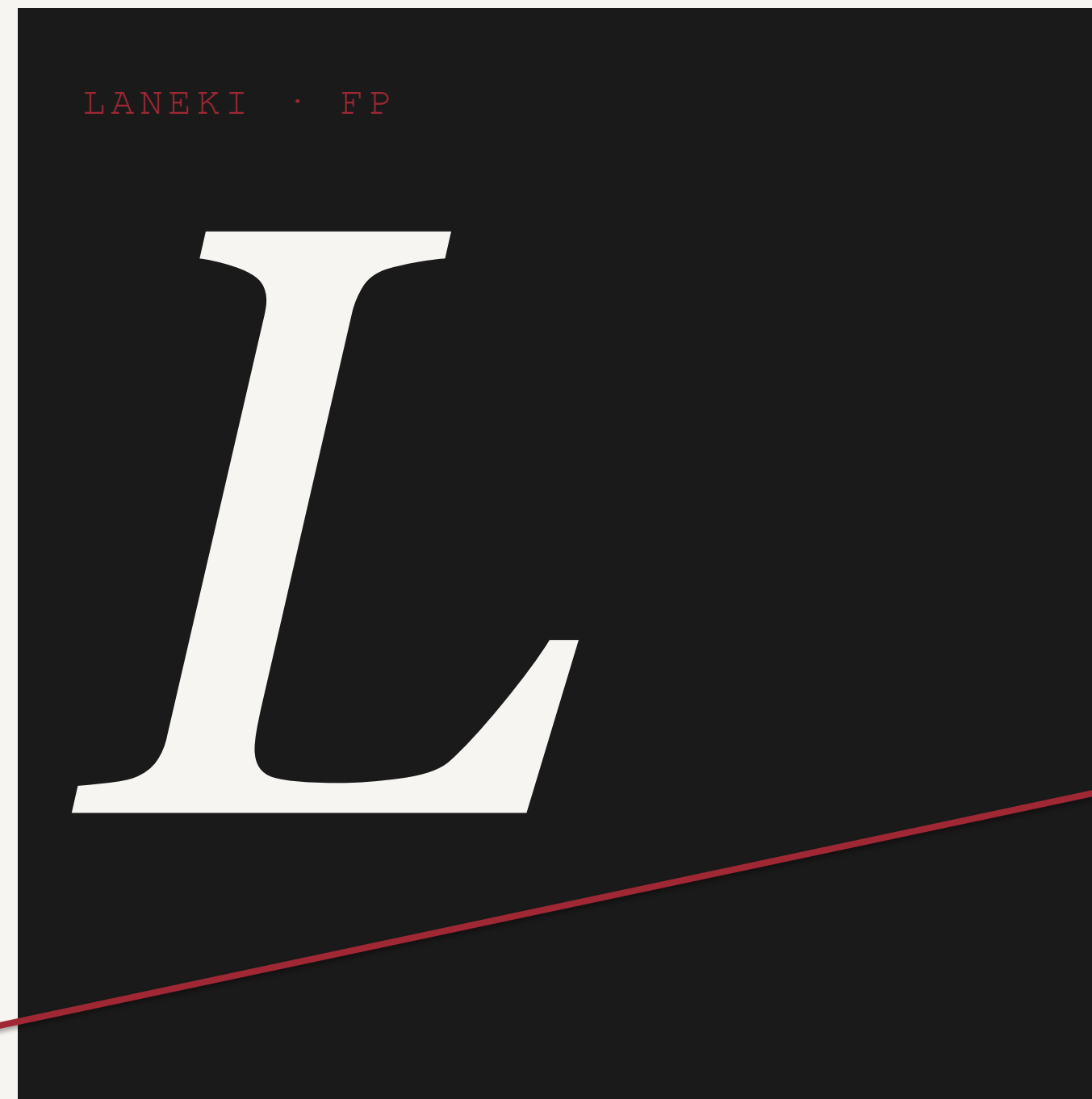


Apache *Kafka*

Denbora errealeko gertaera-streaming plataforma: topikak, brokerrak, kontsumitzaile-taldeak eta Python integrazioa.

Zikloa Adimen Artifiziala eta Big Data

Modulua Big Data Aplikatua



01 DATAFLOW. Apache Kafka

01 Sarrera — zer da Kafka, zergatik eta noiz

02 Argitaratu-Harpidetu eredua (Pub/Sub)

03 O. kasua — Kafka kaixo (zerbitzuak + komando-lerroa)

04 Elementuak — topic-ak, partizioak, brokerrak, errepikatze

05 Ekoizleak — ACK konfigurazioa, mezu-gakoak

06 Kontsumitzaileak eta kontsumitzaile-taldeak

07 Broker Discovery

08 Kafka bermeak

09 1. kasua — Kafka Python-ekin integratuta

01 Sarrera

Apache Kafka

Apache Kafka

Denbora errealeko **GERTAEREN STREAMINGA** egiteko plataforma **BANATU** eta **KODE IREKIKOA** da. Gertaerak modu **IRAUNKORREAN** gordetzeko eta sistema desberdinen artean banatzeko aukera ematen du.

MIDDLEWARE gisa funtzionatzen du, **EKOIZLEAK (Producers)** eta **KONTSUMITZAILEAK (Consumers)** elkarrengandik bereiziz.

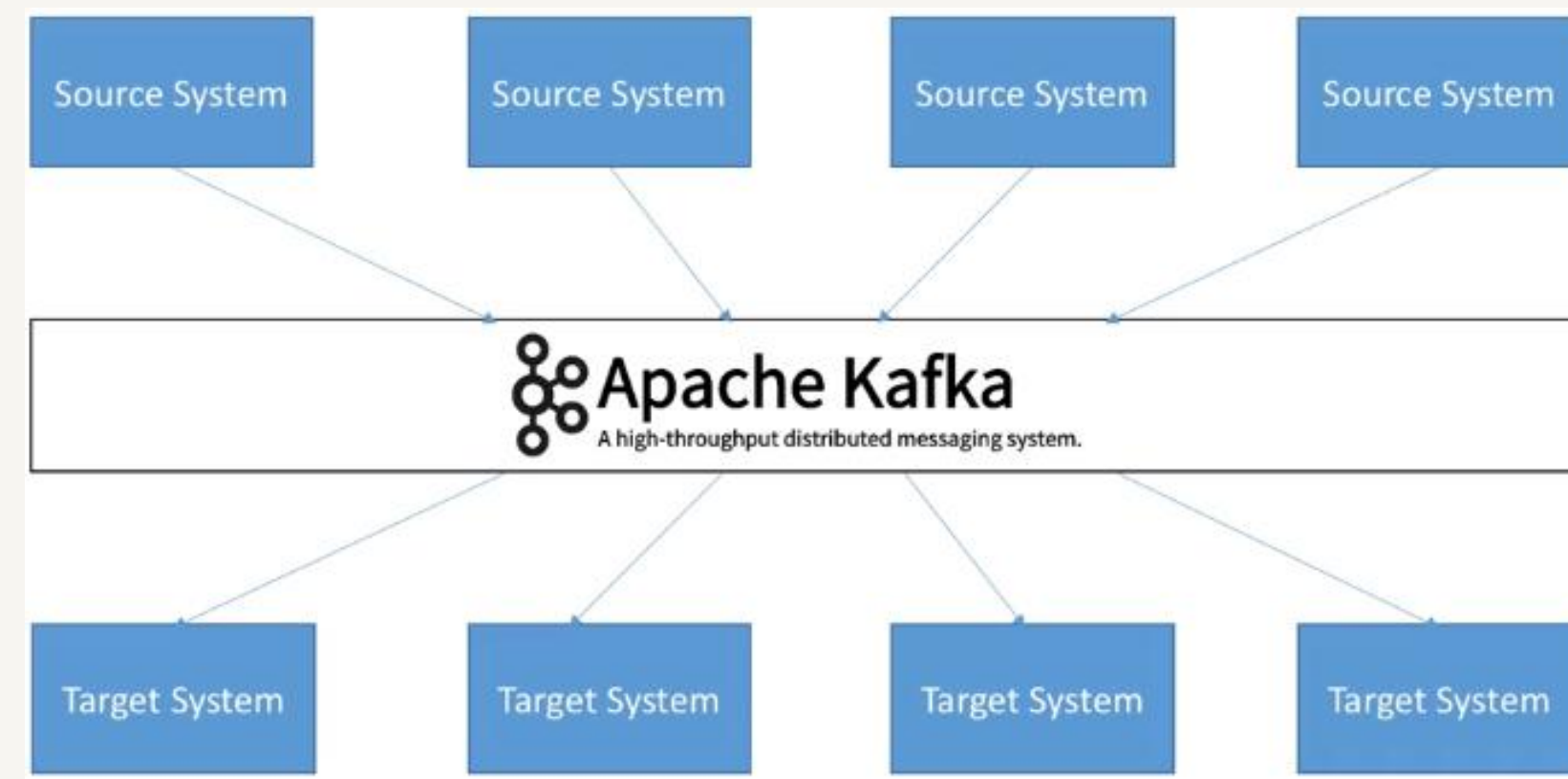
MIDDLEWARE-A

Ekoizleak eta kontsumitzaileak **ELKARRENGANDIK ASKEAK** dira: ekoizleak ez du jakin behar nork kontsumituko dituen bere datuak; kontsumitzaileak bere erritmoan irakur ditzake Kafka-n oraindik gordeta dauden gertaerak.

— MIDDLEWARE-A

Kafka middleware gisa — sistemen arteko zubi

- **GOIKO ALDEAN: ITURRI-SISTEMAK** (web aplikazioak, mugikorreko aplikazioak, IoT gailuak, mikrozerbitzuak...)
- **BEHEKO ALDEAN: KONTSUMITZAILE-SISTEMAK** (Data Warehouse, analitika, Machine Learning, auditoria...)
- **ERDIAN: KAFKA** — iturriak eta kontsumitzaileak Kafka-ra konektatzen dira
- **KAFKA GABE:** Producer eta Consumer-en arteko integrazio zuzen ugari (N x M arte)
- **KAFKA-REKIN:** sistema bakoitza Kafka-ra konektatzen da (N+M integrazio): arkitektura sinpleagoa eta mantentzeko errazagoa



Ezaugarri nagusiak eta erabilera-kasuak

EZAUGARRI NAGUSIAK:

- **THROUGHPUT HANDIA ETA LATENTZIA BAXUA**
— gertaera-fluxu oso handiak denbora errealean kudeatzeko diseinatua
- **AKATS-TOLERANTZIA + ESKALAGARRITASUN HORIZONTALA + ERABILGARRITASUN HANDIA**
- **ESKALA HANDIKO PRODUKZIO-INGURUNEETAN OSO ERABILIA** — LinkedIn, Uber, Netflix, Spotify...

ERABILERA KASUAK:

- **MEZULARITZA ETA DATU-INGESTIOA** — aplikazio, log, clickstream eta bestelako gertaerak jasotzea
- **STREAMING PROZESAMENDUA ETA DENBORA ERREALEKO ANALITIKA**
- **MIKROZERBITZUAK** — event-driven komunikazioa
- **IoT** — sentsoire eta gailuetatik datozen gertaera-fluxuak kudeatzea

02

Argitaratu- Harpidetu eredua

Argitaratu-Harpidetu (Pub/Sub)

Pub/Sub eredua

Hiru elementu nagusi ditu:

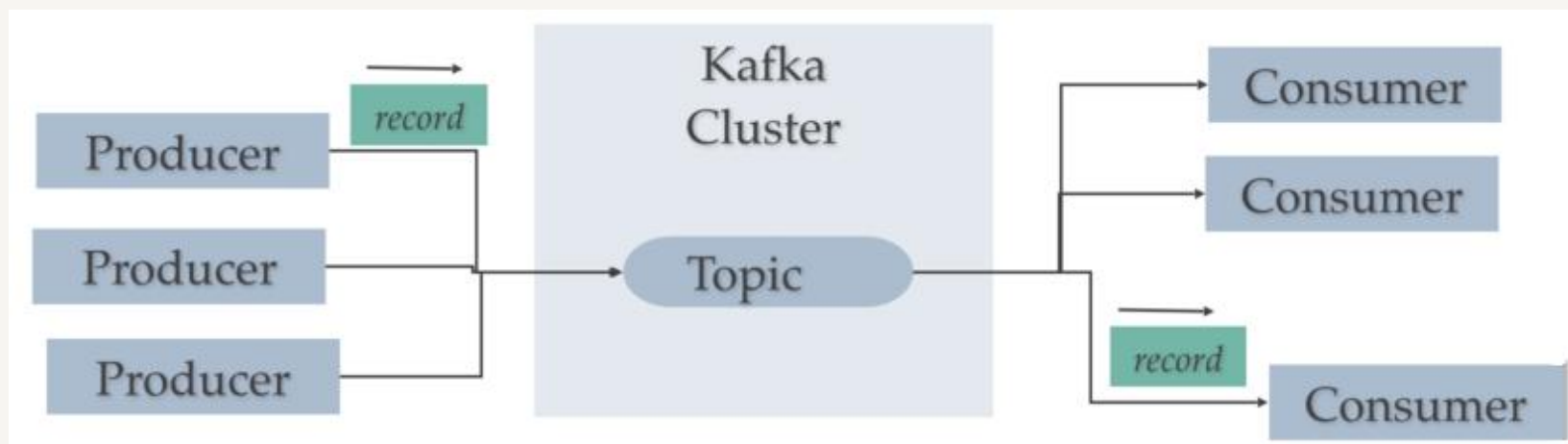
- **EKOIZLEA (Producer)**: gertaerak/mezuak sortu eta **topic** batera bidaltzen ditu.
- **TOPIC-A (Topic)**: mota bereko gertaerak/mezuak antolatu eta gordetzen dituen **kanal** logikoa da.
- **KONTSUMITZAILEA (Consumer)**: topic bateko gertaerak/mezuak irakurtzen ditu.

BEREIZIAK

Producer-a EZ da zuzenean Consumer-arekin komunikatzen: dena Topic-en bidez egiten da. BEREIZITA DAUDE: Producer-ak Topic-a ezagutzen du eta Consumer-ak Topic-a ezagutzen du, baina ez dute elkar ezagutu behar. Pub/Sub ereduaren abantaila nagusia: DESAKOPLAMENDUA.

— PUB/SUB ESKEMA

Producer → Topic → Consumer fluxu basikoa



- **PRODUCER-AK** mezua **TOPIC** batera bidaltzen du (send)
- **KAFKA-K** mezua topic-ean gordetzen du
- **CONSUMER-AK** topic-etik mezuak irakurtzen ditu (poll), bere erritmoan
- Topic berean **hainbat Producer eta Consumer** egon daitezke.
- **Mezua ez da ezabatzen irakurtzean** — retention-denboran topic-ean gordeta jarraitzen du

03

0. Kasua — Kafka Kaixo

0. KASUA – Kafka Kaixo

- **HELBURUA:** Kafka Docker-en martxan jarri, iabd-topic izeneko topic bat sortu, Producer batetik mezuak bidali eta Consumer batean jasotzen direla ikusi.
- Amaieran hau izango dugu:

Producer → iabd-topic → Consumer

1. Kafka Docker bidez abiarazi

1. Sortu ariketaren karpeta:
C:\kafka\ariketak\0-kasua-kafka-kaixo
2. Sortu **compose.yaml** fitxategia karpeta horretan:

```
C: > kafka > ariketak > 0-kasua-kafka-kaixo > compose.yaml
1  services:
2    kafka:
3      image: apache/kafka:latest
4      container_name: iabd-kafka
5      ports:
6      - "9092:9092"
7      environment:
8      - PATH: "/opt/kafka/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
9
```

3. Abiarazi Kafka:
docker compose up -d
4. Egiaztatu container-a martxan dagoela (espero den container-a: *iabd-kafka*)
docker ps

2. Topic-a sortu

1. Sartu Kafka container-ean:

docker exec -it iabd-kafka bash

2. Sortu *iabd-topic*:

kafka-topics.sh --create --topic iabd-topic --bootstrap-server localhost:9092

3. Topic-ak zerrendatu:

kafka-topics.sh --list --bootstrap-server localhost:9092

4. Topic-a deskribatu:

kafka-topics.sh --describe --topic iabd-topic --bootstrap-server localhost:9092

--bootstrap-server: edozein broker (kluster osoaren metadatuak eskuratzeko gako-konexioa)

3. Producer-a – Mezuak bidali

Ireki beste PowerShell terminal bat:

1. Sartu Kafka container-ean:

docker exec -it iabd-kafka bash

2. Abiarazi Producer-a:

kafka-console-producer.sh --topic iabd-topic --bootstrap-server localhost:9092

3. Bidali mezu batzuk (lerro bakoitza mezu bat da):

> **Kaixo Kafka**

> **Hau nire lehen mezua da**

> **Big Data Aplikatua**

4. Consumer-a – Mezuak irakurri

Ireki beste PowerShell terminal bat:

1. Sartu Kafka container-ean:

docker exec -it iabd-kafka bash

2. Abiarazi Producer-a:

kafka-console-consumer.sh --topic iabd-topic --from-beginning --bootstrap-server localhost:9092

Emitza:

Kaixo Kafka

Hau nire lehen mezua da

Big Data Aplikatua

5. Streaming-a probatu

Utzi **Producer** eta **Consumer** terminalak biak martxan.

1. Producer-ean bidali mezu batzuk (lerro bakoitza mezu bat da):

- > Laugarren mezua
- > Bosgarren mezua

Zer ikusi dugu?

- Producer-ak mezuak **Topic-era bidaltzen ditu**
- Kafka-k mezuak **gordetzen ditu**
- Consumer-ak mezuak **irakurtzen ditu**
- Producer eta Consumer-ek **aldi berean** lan egin dezakete
- Consumer-ak mezu bat irakurtzeak **EZ du mezua ezabatzen**

6. *--from-beginning* probatu eta amaitu

1. Gelditu Consumer-a: **Ctrl + C**

2. Berriro abiarazi Consumer-a:

kafka-console-consumer.sh --topic iabd-topic --from-beginning --bootstrap-server localhost:9092

Zer gertatzen da?

Aurreko mezuak **berriro agertzen dira**

Beraz: **IRAKURRI ≠ EZABATU**

Kafka-k mezuak Topic-ean gordetzen jarraitzen du **retention-politikaren arabera**

3. Amaitzeko: **Ctrl + C**

4. Container-etik atera: **exit**

5. Kafka gelditu:

docker compose down

--from-beginning: HASIERATIK irakurri (offset 0)
--from-beginning gabe: uneko mezu BERRIAK soilik

04

Kafka-ren barne-arkitektura

Topic-a eta Partizioa

Topic + Partition

TOPIC-A: datu-korrante logiko bat da. Topic bakoitza PARTIZIO batean edo gehiagotan banatzen da.

PARTIZIOA: topic baten datuak gordetzen dituen commit-log ordenatua da. Partizioak 0-tik hasita zenbakitzen dira.

Mezu bakoitzak **OFFSET** bat du: partizioaren barruan duen posizioa.

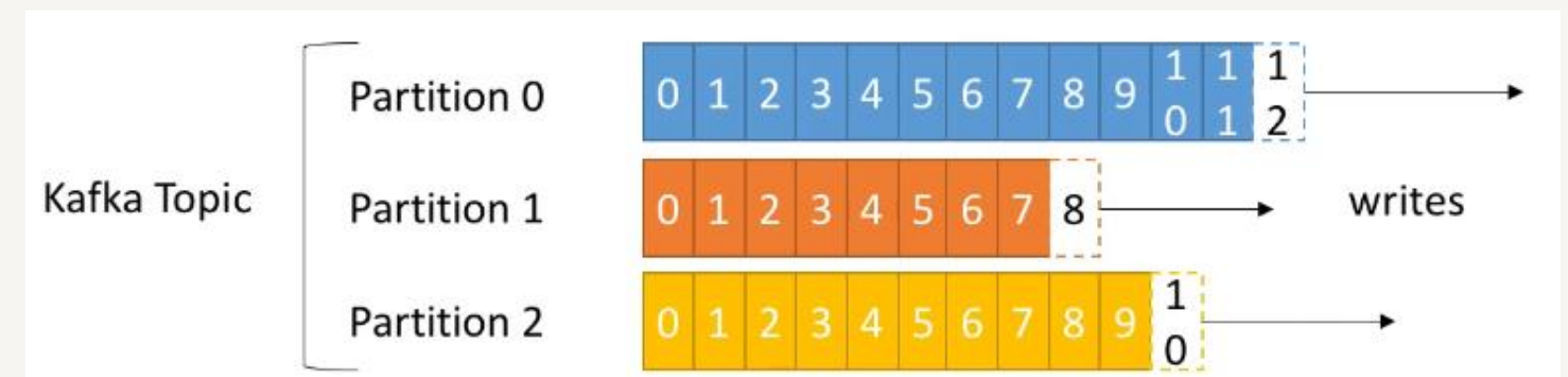
PARALELISMOA

PARTIZIOA = paralelismo-unitatea. Topic batek **N** partizio baditu → consumer group bereko gehienez **N** kontsumitzailek irakur dezakete **paraleloan**.

— PARTIZIOAK

Topic-a N partiziotan — ordena BARRUAN bermatua

- **TOPIC** bat (adib. *iabd-topic*) = **N partizio** (0, 1, 2...)
- **Partizio bakoitza** = ordenatutako **COMMIT-LOG** bat (*append-only*)
- **MEZU bakoitza partizio batera doa** — *key*-aren arabera edo, *key*-rik ez badago, producer-ak partizioen artean banatzen du (*adib. round-robin*)
- **ORDENA PARTIZIOAREN BARRUAN** bermatzen da — partizioen artean **EZ**
- **OFFSET** = mezuak partizioaren barruan duen **posizioa** (0, 1, 2...)



Topic-en eta partizioen ezaugarri nagusiak

- **OFFSET PARTIZIO-MAILAN dago** — partizio bakoitzak bere kontagailua du (0, 1, 2...)
- **ORDENA PARTIZIO BARRUAN** baino ez da bermatzen (ez topic osoan)
- **RETENTION konfiguragarria da** — mezuak denbora/tamaina jakin batera arte gordetzen dira
- **MEZUAK ALDAEZINAK dira** behin idatzita — *append-only* log eredua
- **PARTIZIO-ESLEIPENA** — **key**-arekin deterministikoa; **key**-rik gabe producer-ak partizioen artean banatzen ditu

Topic-a partizioekin sortu, aztertu eta ezabatu

1. Sartu Kafka container-ean:
docker exec -it iabd-kafka bash
2. 3 partizioko topic-a sortu:
kafka-topics.sh --create --topic iabd-topic-p3 --partitions 3 --bootstrap-server localhost:9092
3. Topic-a deskribatu:
kafka-topics.sh --describe --topic iabd-topic-p3 --bootstrap-server localhost:9092
4. Topic-a ezabatu:
kafka-topics.sh --delete --topic iabd-topic-p3 --bootstrap-server localhost:9092

OHARRA: Partizio kopurua gerora **handitu** daiteke, baina **ezin da murriztu**.

1. ARIKETA – Topic berriak sortu eta kudeatu

Sortu honeko bi topic-ak:

- **sentsoreak**
- **erosketak**

Baldintzak:

- **sentsoreak** → 2 partizio
- **erosketak** → 4 partizio

Erabili 0. kasuan ikasitako **kafka-topics.sh –create** komandoa, behar diren parametroak egokituz.

Ondoren:

1. Erakutsi Kafka-n dauden topic guztiak.
2. Egin –describe bi topic-ei.
3. Egiaztatu bakoitzak eskatutako partizio kopurua duela.
4. Identifikatu partizio bakoitzaren zenbakia

2. ARIKETA – Producer eta Consumer berriak

Erabili aurreko ariketako **sentsoreak** topic-a.

Ireki bi terminal.

1. terminalean: sortu sentsoreak topic-erako producer bat.

Bidali gutxienez mezu hauek:

- Eibar;21.4
- Ermua;20.8
- Eibar;21.7
- Markina;19.6
- Ermua;21.0
- Eibar;22.1

2. terminalean: sortu topic horren consumer bat.

Ondoren:

1. Itxi consumer-a
2. Producer-etik bidali beste hiru neurketa
3. Consumer-a berriro martxan jarri **--from-beginning** aukerarekin. Aztertu: Consumer-a deskonektatuta egon den bitartean bidalitako mezuak galdu dira?

3. ARIKETA – Mezuak ez dira irakurtzean ezabatzen

Erabili **erosketak** topic-a.

Producer batetik bidali: **E001;ordenagailua;899**
E002;monitorea;249
E003;teklatua;59
E004;SSD;89
E005;routerra;119

Ondoren, ireki consumer bat **--from-beginning** erabiliz eta irakurri mezu guztiak.

Consumer-a itxi.

Ireki **beste consumer bat** berriro **--from-beginning** erabiliz.

Galdera: Bigarren consumer-ak lehenengo consumer-ak irakurritako mezuak irakur ditzake?

4. ARIKETA – Partizioa eta offset-a ikusi

Erabili aurreko ariketako **erosketak** topic-a.

erosketak topic-ak 4 partizio ditu eta dagoeneko hainbat mezu ditu.

Consumer-a exekutatzean, gehitu informazio hau erakusteko aukerak: **--property print.partition=true**
--property print.offset=true
Irakurri mezuak hasieratik.

Emaitzak aztertu eta bete antzeko taula bat:

Mezua	Partition	Offset
E001...		
E002...		
E003...		
E004...		
E005...		

Ondoren, erantzun:

1. Partizio guztiek offset kontagailu bera partekatzen dute?
2. Offset=0 behin baino gehiagotan ager daiteke?
3. Nahikoa al da offset-a bakarrik mezu bat identifikatzeko?
4. Zer bi datu behar ditugu?

5. ARIKETA – Partizio kopurua handitu

Erabili **sentsoreak** topic-a, hasieran **2 partiziorekin** sortu duguna.

1. Egiaztatu **--describe** erabiliz 2 partizio dituela.
2. Bilatu 0. kasuan erabilitako **kafka-topics.sh** komandoaren aukerak eta aldatu topic-a, **4 partizio** izan ditzan.
3. Egin berriro **--describe**.
4. Egiaztatu orain 4 partizio daudela.

Ondoren saiatu: 4 partiziotik **2 partiziora** jaistea.

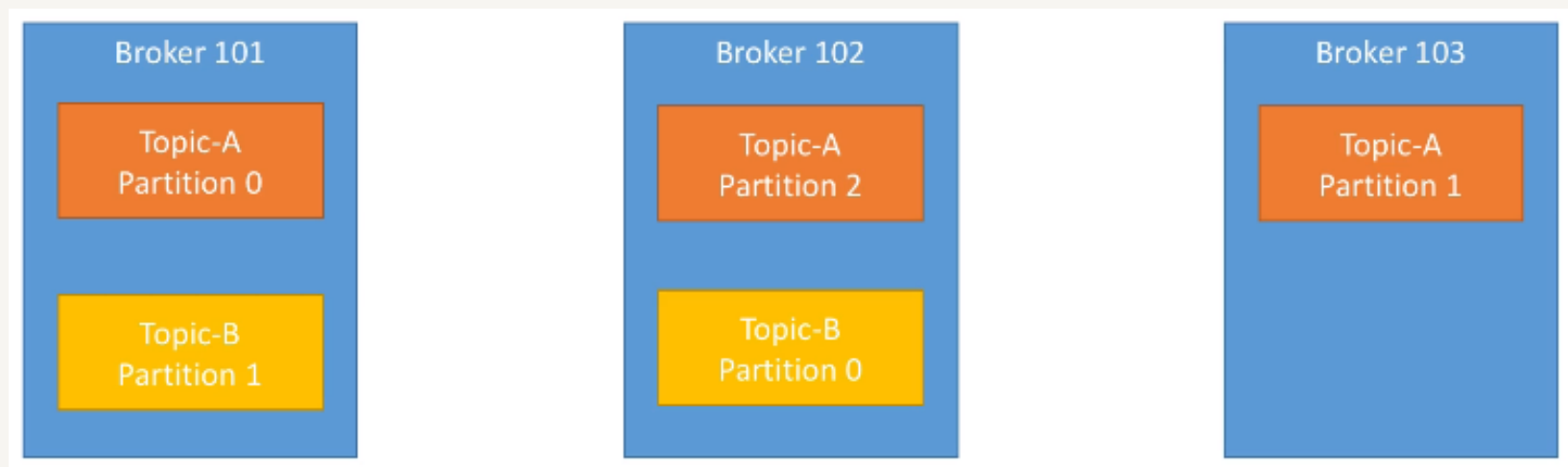
Broker

Broker

Kafka kluster bat **BROKER** deritzon hainbat nodok osatzen dute, eta bakoitzak **ID bakarra** du. Broker bakoitzak topic-en **PARTIZIOAK gordetzen ditu**. Producer edo consumer bat **BOOTSTRAP broker** batera konektatzen da, klusterraren informazioa lortzeko eta dagokion brokerrarekin komunikatzeko.

BANATUA ESKALAGARRIA

Broker bat = Kafka zerbitzari bat. Topic-en **PARTIZIOAK broker-en artean banatzen dira**. Gainera, partizioen **ERREPLIKAK** broker desberdinetan gorde daitezke; horri esker, broker batek huts egiten badu, **sistemak erabilgarri jarrai dezake**.



— KLUSTER BANATUA

3 broker-eko Kafka kluster baten adibidea

- **Broker 1, Broker 2, Broker 3** — bakoitza Kafka zerbitzari independentea
- **PARTIZIOAK broker-en artean BANATZEN dira** — broker bakoitzak partizio batzuk gordetzen ditu
- **BOOTSTRAP broker batera konektatuz** → kluster osoaren metadatuak lortzen dira
- **ERREPLIKAZIOARI esker**, broker batek huts eginda ere zerbitzuak erabilgarri jarrai dezake
- Produkzioan **3 broker-eko klusterra ohiko abiapuntua** da high availability lortzeko

04b

Errepikatze- faktorea

Errepikatze-faktoredun topic-a sortu

1. 2 broker edo gehiagoko kluster batean:

```
kafka-topics.sh --create --topic TopicA --partitions 2 --replication-factor 2 --bootstrap-server localhost:9092
```

2. Eraitza aztertzeko:

```
kafka-topics.sh --describe --topic TopicA --bootstrap-server localhost:9092
```

```
Adibidez: Partition: 0    Leader: 0    Replicas: 0,1    Isr: 0,1
           Partition: 1    Leader: 1    Replicas: 1,0    Isr: 1,0
```

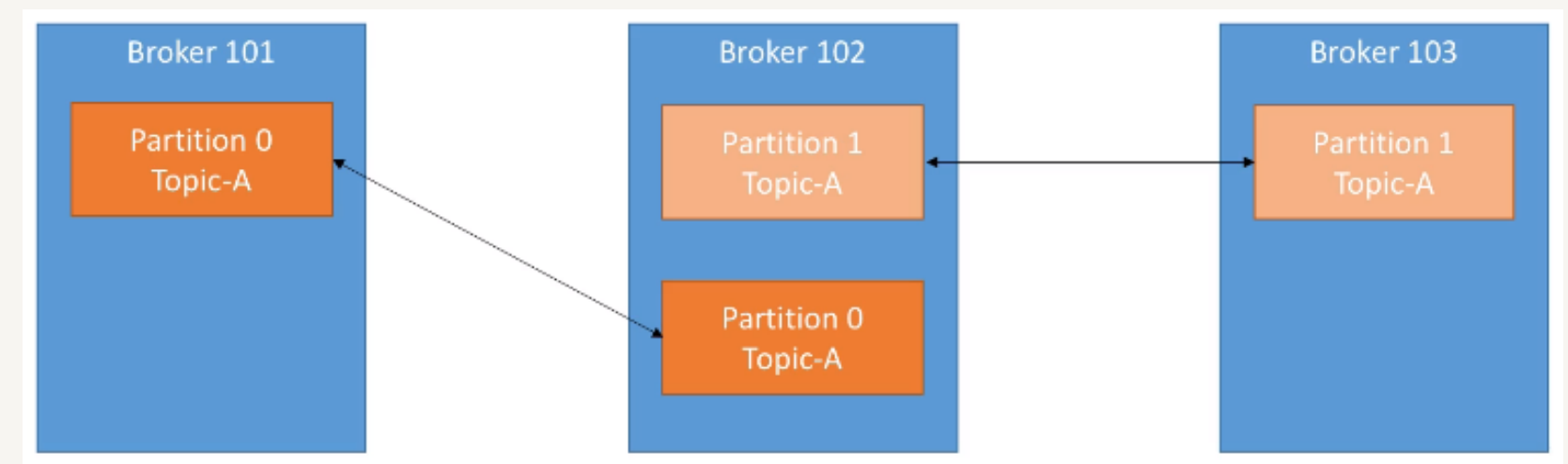
Interpretazioa:

- Replication Factor = 2 → partizio bakoitzaren 2 kopia
- Leader → partizioaren kopia nagusia
- Replicas → kopia gordetzen duten broker-ak
- ISR → une horretan sinkronizatuta dauden erreplikak
- Araua: replication factor ≤ broker kopurua

— ERREPLIKAK

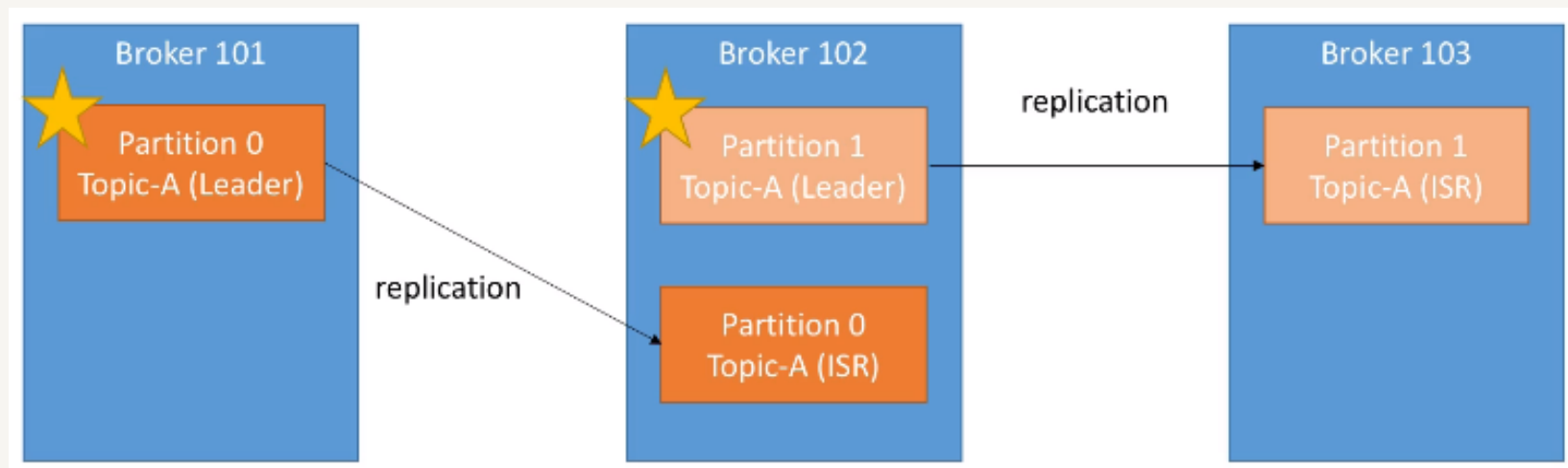
Errepikatze-faktorea — partizioak hainbat brokerretan

- **PARTIZIO** bakoitzak **N kopia** ditu, **replication factor = N** denean
- Partizio bakoitzak **LEADER** bakarra du — eskaerak leader-ak kudeatzen ditu
- Beste kopiak **FOLLOWER REPLICAS** dira — leader-arekin sinkronizatzen dira
- **FAULT-TOLERANCE** — erreplikek broker baten hutsegitearen aurrean erabilgarritasuna mantentzea ahalbidetzen dute.
- **ERREPLIKAK** broker desberdinetan banatzen dira



— ERREPLIKAK BANAKETA

Partizioen erreplikak broker-en artean



- Topic-aren partizioak **BROKER ANITZETAN ERREPLIKATZEN** dira (**replication-factor**>1)
- **LEADER ETA FOLLOWER** erreplikak broker desberdinetan gordetzen dira
- **ISR (In-Sync Replicas)** = leader-arekin sinkronizatuta dauden erreplikak
- Leader-a duen broker-ak huts egiten batu, **ISR-ko** beste erreplika batek **LEADER** rola har dezake
- **FAILOVER AUTOMATIKOA** → zerbitzuaren erabilgarritasuna mantentzen laguntzen du

Leader erreplika eta ISR (*In-Sync Replicas*)

- **PARTIZIO BAKOITZAK LEADER BAT** du — partizioaren eskaerak kudeatzen ditu
- **FOLLOWER REPLIKAK** — leader-aren datuak etengabe erreplikatzen dituzte
- **ISR (IN-SYNC REPLICAS)** — leader-a + sinkronizatuta dauden follower-ak
- **acks=all + min.insync.replicas=2** — produkzioan ohiko konfigurazioa, datuen fidagarritasuna handitzeko
- **LEADER-ak huts egiten badu** → ISR-ko beste erreplika bat **LEADER** bihur daiteke (*failover*)

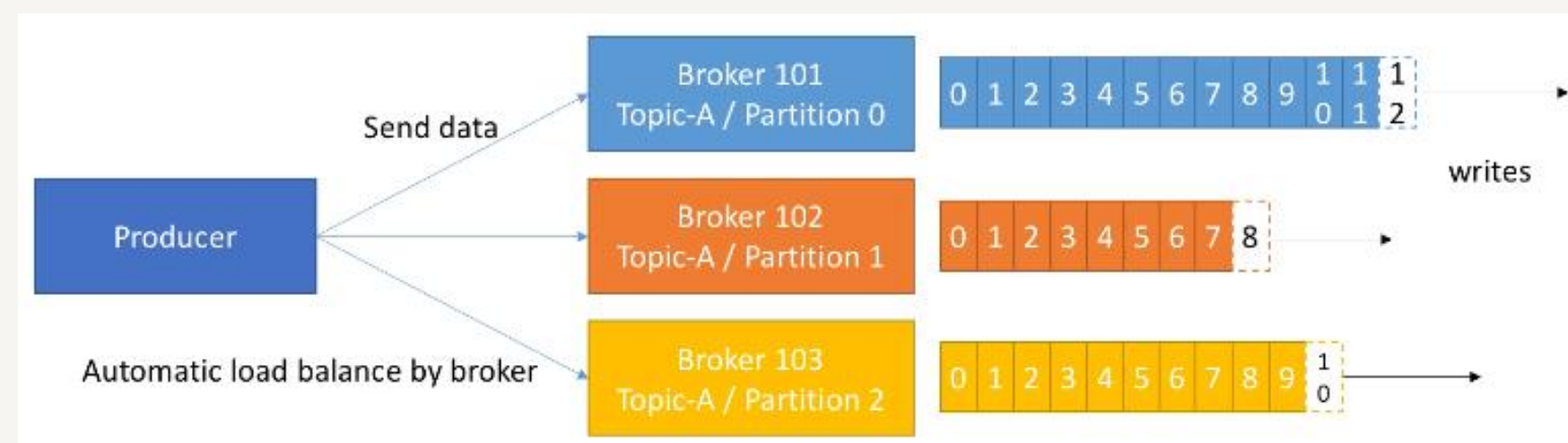
05

Ekoizleak (Producers)

— KARGA BANAKETA

Ekoizleak mezuak partizioen artean banatzen ditu

- **GAKORIK GABE** — producer-ak mezuak partizioen artean banatzen ditu, **karga orekatzeko**
- **GAKOAREKIN** — gako berak **partizio bera** aukeratzen du modu deterministan
- **GAKOA → ORDENA** mantentzea ahalbidetzen du entitate-mailan (kontu-ID, sentso-re-ID...)
- Producer-ak partizioaren **LEADER broker-a automatikoki aurkitzen du**, metadatuaren bidez
- Broker/leader aldaketa batean → producer-ak **metadatuak eguneratu eta berriro saiatzzen da**



ACK konfigurazioa — durabilitate-aukerak

- **acks=0** — Berrespenik EZ → azkarrena, baina **datu-galera posiblea**
- **acks=1** — LEADER-ak baieztatzen du → berme handiagoa, baina leader-a galtzean **datu-galera posiblea**
- **acks=all (-1)** — erreplikazioaren berrespena → **durabilitate handiena** **min.insync.replicas** parametroarekin batera erabiltzen da
- **MEZU-GAKOA** — gako bera → partizio bera → entitate-mailako ordena
- **KONPRESIOA** — **gzip**, **snappy**, **lz4**, **zstd** → sare-trafikoa eta biltegitratzea murrizteko

Console Producer mezu-gakoekin

1. Producer-a abiarazi:

```
kafka-console-producer.sh --topic iabd-topic --property "parse.key=true" --property "key.separator=:" --bootstrap-server localhost:9092
```

2. Bidali mezuak **gakoa:balioa** formatuan:

```
gakoa1:balioa1
gakoa2:balioa2
gakoa1:balioa3
bezero_1:{"bezero_id":1,"izena":"Aitor","abizena":"Medrano"}
```

3. Consumer-ean gakoak ere ikusi:

```
kafka-console-consumer.sh --topic iabd-topic --from-beginning --property print.key=true --property key.separator=":" --bootstrap-server localhost:9092
```

OHARRAK:

- **parse.key=true** → producer-ak gakoa interpretatzen du
- **print.key=true** → consumer-ak gakoa erakusten du
- **Gako bera** → **partizio bera** → **ordena mantentzeko aukera**

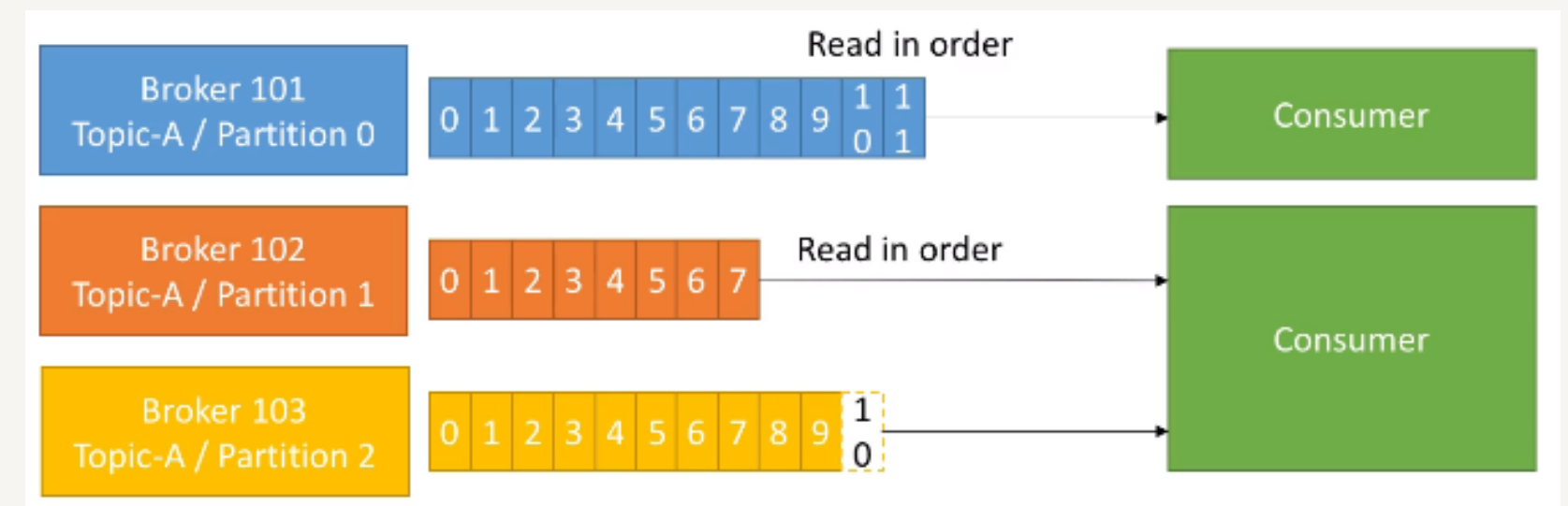
06

Kontsumitzaileak eta taldeak

— KONTSUMITZAILEAK

Kontsumitzaileak partizioetatik irakurtzen

- Consumer-ek **TOPIC-aren PARTIZIOETATIK** irakurtzen dituzte mezuak
- Mezuak **OFFSET-ORDENAN** jasotzen dira partizio bakoitzean (0, 1, 2...)
- **OFFSET-en jarraipena** → consumer-ak badaki nondik jarraitu behar duen
- **POLL eredua** → consumer-ak datuak eskatzen dizkio Kafka-ri (*pull*)
- Consumer batek **PARTIZIO BAT EDO GEHIAGO** irakur ditzake



Kontsumitzaile-taldea (Consumer Group)

Consumer Group

Talde bateko **KONTSUMITZAILEEK** topic baten **PARTIZIOAK** beraien artean banatzen dituzte. Partizio bakoitza taldeko **consumer bakar bati** esleitzen zaio une bakoitzean. Talde **DESBERDINEK** topic bereko datuak **modu INDEPENDENTEAN** kontsumitzen dituzte. Consumer kopurua partizio kopurua baino handiagoa bada, consumer batzuk **INAKTIBO** geratzen dira.

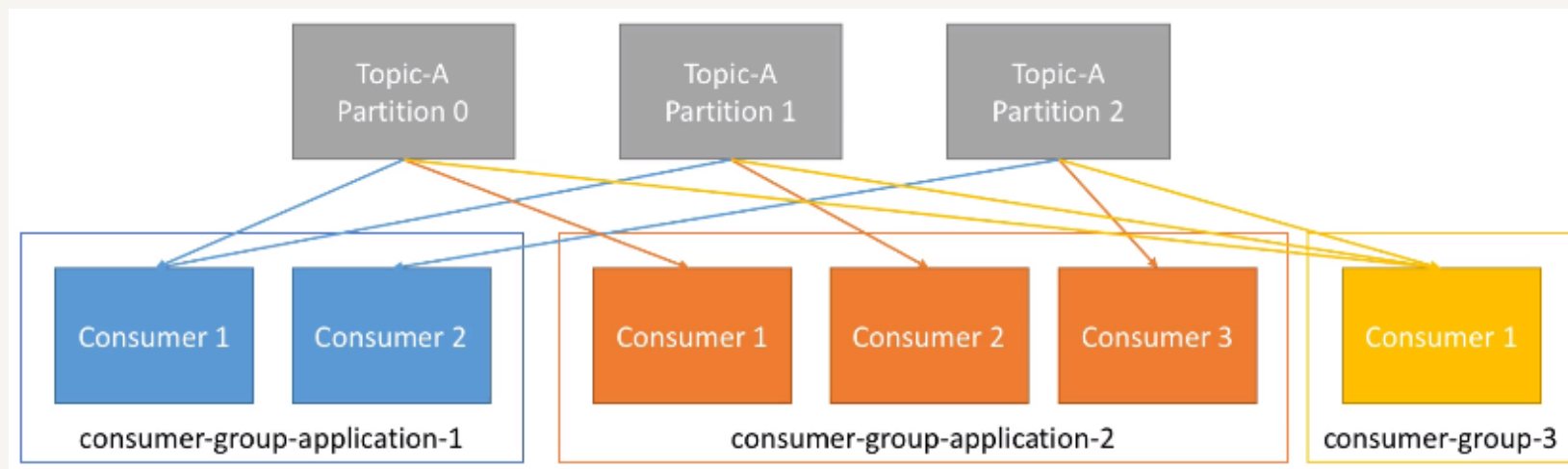
PARALELISMOA

Talde berekoak → LANA BANATZEN dute (partition-banaketa)

Talde ezberdinekoak → DATU BERAk independenteki kontsumitzen dituzte

— CONSUMER GROUP

Kontsumitzaile-taldea — partizio-banaketa



- **TOPIC BAT** → N partizio (**P0**, **P1**, **P2...**)
- **CONSUMER GROUP BAT** → M kontsumitzaile (**C1**, **C2...**)
- **M ≤ N** → consumer bakoitzak partizio bat edo gehiago jasotzen ditu
- **M > N** → soberako consumer-ak **INAKTIBO** geratzen dira (*idle*)
- **group.id** → consumer-ak talde berean identifikatzeko

Talde anitzak — independentzia

```
iabd@iabd-virtualbox:/opt/kafka_2.13-2.8.1$ kafka
-console-consumer.sh --topic iabd-topic-group \
> --group iabd-app1 \
> --bootstrap-server iabd-virtualbox:9092
Prueba 1
Prueba 3
Prueba 8
Prueba 10
□

iabd@iabd-virtualbox:/opt/kafka_2.13-2.8.1$ kafka-console-consumer.sh --topic iabd
-topic-group \
> --group iabd-app1 \
> --bootstrap-server iabd-virtualbox:9092
Prueba 2
Prueba 4
Prueba 5
Prueba 6
Los mensajes se reparten entre los consumidores del mismo grupo
Prueba 9
□

iabd@iabd-virtualbox:/opt/kafka_2.13-2.8.1$ kafka-console-producer.sh --topic iabd
-topic-group --bootstrap-server iabd-virtualbox:9092
>[2022-02-22 17:10:22,631] WARN [Producer clientId=console-producer] Bootstrap broker iabd-virtualbox:9092 (id: -1 rack: null) disconnected (org.apache.kafka.clients.NetworkClient)
Prueba 1
>Prueba 2
>Prueba 3
>Prueba 4
>Prueba 5
>Prueba 6
>Los mensajes se reparten entre los consumidores del mismo grupo
>Prueba 8
>Prueba 9
>Prueba 10
```

- **TALDE A** (**analytics**) — consumer-ek partizioak beraien artean banatzen dituzte
- **TALDE B** (**audit**) — bere consumer-en artean egiten du banaketa
- **BI TALDEEK topic bereko MEZUAK kontsumitzen dituzte** — bakoitzak bere offset-ekin
- Talde bateko consumer bat erortzean → **talde horretan bakarrik** egiten da *rebalance*
- Helburu-ezberdinak izan ditzakete: **A = analytics; B = audit**

Bi consumer talde berean — partizio-banaketa

... taldea-praktikan.sh

bash

```
1 # 1) 3 partiziozko topic-a sortu
2 kafka-topics.sh --create --topic iabd-topic-taldea \
3   --partitions 3 \
4   --bootstrap-server iabd-virtualbox:9092
5
6 # 2) TERMINAL 1 - Lehen kontsumitzailea 'iabd-app1' taldean
7 kafka-console-consumer.sh --topic iabd-topic-taldea \
8   --group iabd-app1 \
9   --bootstrap-server iabd-virtualbox:9092
10
11 # 3) TERMINAL 2 - Bigarren kontsumitzailea TALDE BEREAN
12 kafka-console-consumer.sh --topic iabd-topic-taldea \
13   --group iabd-app1 \
14   --bootstrap-server iabd-virtualbox:9092
15
16 # 4) TERMINAL 3 - Ekoizlea
17 kafka-console-producer.sh --topic iabd-topic-taldea \
18   --bootstrap-server iabd-virtualbox:9092
19
20 # EMAITZA: 3 partizio + 2 consumer
21 #   -> 1 consumer-ek 2 partizio, beste consumer-ak 1 partizio
22 #   -> mezuak BANATUTA agertuko dira terminal bietan
```

kafka-consumer-groups.sh — list eta describe

```
●●● kudeatu-taldeak.sh bash

1 # 1) TALDE GUZTIAK ZERRENDATU
2 kafka-consumer-groups.sh --list \
3   --bootstrap-server iabd-virtualbox:9092
4
5 # 2) TALDE ZEHATZA DESKRIBATU (egoera eta offset-ak)
6 kafka-consumer-groups.sh --describe --group iabd-app1 \
7   --bootstrap-server iabd-virtualbox:9092
8
9 # 3) OFFSET MANUALA RESET-egin (etenduran)
10 kafka-consumer-groups.sh --reset-offsets \
11   --group iabd-app1 --topic iabd-topic-taldea \
12   --to-earliest --execute \
13   --bootstrap-server iabd-virtualbox:9092
14
15 # OHARRAK:
16 #   --reset-offsets: consumer-talde GUZTIA gelditu behar
17 #                   (offsets aktiboak ez aldatzeko)
18 #   --to-earliest: hasieratik berriz irakurri
19 #   --to-latest: uneko bukaerara salto egin
20 #   --to-datetime: data zehatz batera
```

Talde-egoera — aktibo vs geldituta

●●● lag-emaitza.txt text

```
1 # AKTIBO (kontsumitzaileak dabil tza, lag=0)
2 GROUP      TOPIC      PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
3 iabd-app1  iabd-topic-taldea 0      4              4              0
4 iabd-app1  iabd-topic-taldea 1      2              2              0
5 iabd-app1  iabd-topic-taldea 2      4              4              0
6
7 # GELDITUTA (5 mezu berri etorri dira)
8 GROUP      TOPIC      PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
9 iabd-app1  iabd-topic-taldea 2      4              5              1
10 iabd-app1  iabd-topic-taldea 1      2              4              2
11 iabd-app1  iabd-topic-taldea 0      4              6              2
12
13 # INTERPRETAZIOA:
14 #   AKTIBO: LAG=0 (consumer-ek mezu guztiak prozesatu dituzte)
15 #   GELDITUTA: LAG=1,2,2 -> 5 mezu pilatu dira partizioetan
16 #   BERRIRO ABIATUTAKOAN, lag jaisten hasiko da
```

— OFFSET PERSISTENTEA

Consumer offsets — azken irakurri-puntua



- **__CONSUMER_OFFSETS TOPIC BEREZIAN gordeta** — Kafka-k Kafka-bidez bere koordinazioa
- **ENABLE.AUTO.COMMIT (default true)** — periodikoki automatikoki commit (5 sec)
- **MANUAL COMMIT** — `consumer.commitSync()` edo `consumer.commitAsync()` — kontrol osoa
- **CHECKPOINT GISA** — consumer erortzean, azken commit-tik berriz hasten da
- **CONSUMER-TALDE-MAILAN** — ez consumer-indibidual-mailan (talde-rebalance-ondoren mantenitzen)

Entrega-semantikak — at-most/least/exactly once

- **AT MOST ONCE (gehienez behin)** — mezua jaso, offset-commit AURRETIK; akats-egonez gero MEZUA GALDU
- **AT LEAST ONCE (gutxienez behin)** — mezua prozesatu, offset-commit ONDOREN; akats-egonez gero BIKOIZTUAK
- **EXACTLY ONCE (zehazki behin)** — Kafka Streams + transakzio-API; consumer IDEMPOTENTEA behar
- **AT-LEAST-ONCE produkzio-ohikoena** — consumer-idempotency programatuz
- **EXACTLY-ONCE konplexua** — overhead handia, kasuetan ezinbestean (pago-prozesamendua, etab.)

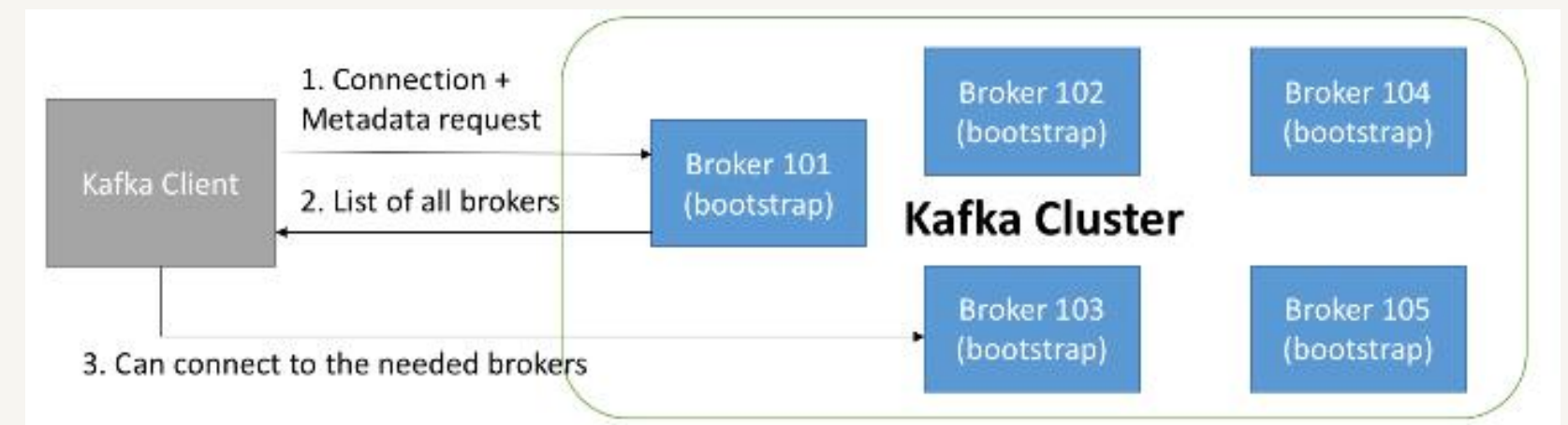
07

Broker
Discovery

— BOOTSTRAP MEKANIKA

Edozein broker = entry-point kluster osora

- EDOZEIN BROKER 'bootstrap-server' gisa balio du
- **Lehen konexioan METADATU-ESKARIA** — kluster osoaren topologia jasotzen
- Erantzunean: broker-id-zerrenda + bakoitzaren host:port + partition-leader-mapping
- GERORA: client-ak ZUZENEAN konektatzen leader-broker-arekin dagokion partizioarentzat
- **BOOTSTRAP-LISTA OHIKOA** — b1:9092,b2:9092,b3:9092 (redundantzia konexio-faseko)



08

Kafka bermeak

Kafka-ren bermeak — sistema-mailako kontratua

- MEZUAK PARTIZIO/TOPIC-EAN BIDALTZE-ORDENAN gehitzen dira (append-only log)
- KONTSUMITZAILEEK GORDEKETA-ORDENAN irakurtzen dituzte mezuak (partizio barruan)
- N ERREPIKATZE-FAKTOREAREKIN → N-1 broker-en hutsegiteak jasan daitezke gabe datu-galera
- **REPLICATION-FACTOR=3 PRODUKZIO-ESTANDAR** — mantentze-leiho + ustekabeko erortze aldi-berean
- **PARTIZIO-KOPURUA EZ ALDATZEN bitartean** — gako BERDINAK partizio BERDINA bermatua (ordena entitate-mailan)

09

1. Kasua — Kafka eta Python

kafka-python — instalazioa

... instalazioa.sh

```
1 # kafka-python liburutegia instalatu
2 pip install kafka-python
3
4 # Egiaztapena Python-en
5 python -c "import kafka; print(kafka.__version__)"
6 # Espero: 2.0.2 edo berriagoa
7
8 # OHARRAK:
9 # 1) Pure-Python inplementazioa (ez du C-konpilaketarik)
10 # 2) Produizioan, confluent-kafka-python hobeagoa:
11 #   - C-binding (librdkafka azpian) — askoz azkarragoa
12 #   - Exactly-once-semantika onartzen
13 #   - Schema Registry integrazioa
14 #   pip install confluent-kafka
15 #
16 # 3) MongoDB-rako ariketa DF3.3-ean ere behar:
17 #   pip install pymongo Faker
```

bash

KafkaConsumer Python-en

●●● kafka_consumer.py

python

```
1 from kafka import KafkaConsumer
2 from json import loads
3
4 # Consumer-aren konfigurazioa
5 consumer = KafkaConsumer(
6     'iabd-topic',                # topic-izena
7     auto_offset_reset='earliest', # hasieratik irakurri
8     enable_auto_commit=True,      # offset-commit auto
9     group_id='iabd-taldea-1',     # talde-izena
10    value_deserializer=lambda m: loads(m.decode('utf-8')),
11    bootstrap_servers=['iabd-virtualbox:9092'])
12
13 # Mezuak etengabe jaso eta inprimatu
14 for m in consumer:
15     print(m.value)
16     # m.topic, m.partition, m.offset, m.key, m.value
17     # m.timestamp ere eskuragarri
```

KafkaConsumer — parametro nagusiak

- **LEHEN POSIZIONALA** — topic-izena (string) edo subscribe([...]) anitzentzat
- **BOOTSTRAP_SERVERS** — broker-zerrenda (list); redundantzia konexio-fasean
- **GROUP_ID** — kontsumitzaile-talde-izena; partition-assignment-aren oinarria
- **AUTO_OFFSET_RESET** — 'earliest' / 'latest' (hasiera-puntua group-offset-ik-ez denean)
- **ENABLE_AUTO_COMMIT** — True (default), background 5s; False = manual commit (kontrol-osoan)
- **VALUE_DESERIALIZER / KEY_DESERIALIZER** — bytes-tik objektu-rako (JSON, Avro, custom)

— 10.3 PRODUCER

KafkaProducer Python-en

••• kafka_producer.py

python

```
1 from kafka import KafkaProducer
2 from json import dumps
3 import time
4
5 # Producer-aren konfigurazioa
6 producer = KafkaProducer(
7     value_serializer=lambda m: dumps(m).encode('utf-8'),
8     bootstrap_servers=['iabd-virtualbox:9092'])
9
10 # 10 mezu bidali, segundo-tarteekin
11 for i in range(10):
12     mezua = {"izena": "ekoizlea " + str(i)}
13     producer.send("iabd-topic", value=mezua)
14     time.sleep(1)
15
16 # producer.flush() # buffer-aren mezuak bidaltzeko (ziurtatu)
17 # producer.close() # konexioak itxi
18
19 # IRTEERA consumer-ean (paraleloan exekutatuz):
20 # {'izena': 'ekoizlea 0'}
21 # {'izena': 'ekoizlea 1'}
22 # ...
23 # {'izena': 'ekoizlea 9'}
```

Gogoan hartu beharrekoak

-
- i* **Pub/Sub eredua** — producer/topic/consumer; ekoizleak eta kontsumitzaileak ELKARRENGANDIK ASKEAK, topic-bidez komunikatzen.
-
- ii* **Partizioak** = paralelismoaren UNITATEA; ordena PARTIZIO BARRUAN bermatua, GAKO-bidez entitate-mailan kontrolatua.
-
- iii* **Replication-factor** = $N \rightarrow N-1$ broker-erortze jasaten; produkzioan 3 estandarra (min-insync-replicas=2, acks=all).
-
- iv* **Consumer groups** — talde-barruan partition-banaketa (paralelizazioa); talde-anitz aldi berean (publish-once-consume-many).
-
- v* **LAG metrikoa** — produkzio-monitorizazio-kritikoa; consumer-osasunaren adierazlea.
-
- vi* **Kafka-Python (kafka-python)** — didaktiko; produkzioan confluent-kafka-python (C-binding, exactly-once, Schema Registry).
-

Galderak, zalantzak, iruzkinak.

APACHE KAFKA DOKUMENTAZIO OFIZIALA	kafka.apache.org/documentation
KAFKA CHEATSHEET (KOMANDOAK)	github.com/lensesio/kafka-cheat-sheet
CONFLUENT DEVELOPER (TUTORIALAK)	developer.confluent.io
JATORRIZKO MATERIALA	Aitor Medrano · CC BY-NC-SA